

Horizon — TUI Smart Destination Recommender

HORIZON: Sistema Inteligente de Recomendación de Destinos Turísticos Sostenibles en España

Trabajo Fin de Máster Máster Universitario en Data Science y Big Data Universidad Complutense de Madrid

Autor: Octavio Álvarez **Correo:** octavio.alvarez@accenture.com **Tutor académico:** TUI Care Foundation — Future Shapers Spain **Convocatoria:** Curso académico 2025–2026

Resumen

El sobreturismo en España representa uno de los principales desafíos medioambientales y sociales de la última década. En 2024, España recibió un récord de 93,8 millones de turistas internacionales, de los cuales aproximadamente el 85% concentró su visita en apenas el 10% del territorio nacional. Esta hipersaturación provoca degradación medioambiental, incremento desmedido del coste de vida y una pérdida progresiva de identidad cultural en los destinos más visitados, mientras que cientos de localidades con un enorme potencial turístico permanecen infravaloradas y subutilizadas.

Horizon es un sistema full-stack de inteligencia artificial diseñado para abordar este problema de raíz: recomendar destinos turísticos sostenibles y menos saturados en España, personalizados según el perfil del viajero y el mes de viaje. El sistema integra un motor de recomendación basado en filtrado por contenido con fórmula de scoring multivariable, un sistema RAG (*Retrieval-Augmented Generation*) conversacional, una API REST construida sobre FastAPI, y una interfaz de usuario reactiva desarrollada con React 19 y Material UI v9.

La fórmula de scoring pondera cuatro dimensiones: preferencia del usuario (45%), sostenibilidad (25%), popularidad (15%) y congestión (15%), con reglas de negocio multiplicativas que penalizan destinos superpoblados y premian aquellos con bajo impacto medioambiental. El sistema opera sobre cinco conjuntos de datos —sintéticos y de fuentes abiertas del INE y FRONTUR— y se distribuye en contenedores Docker para garantizar portabilidad y reproducibilidad.

Los resultados demuestran la viabilidad técnica de aplicar criterios de redistribución turística mediante sistemas de recomendación guiados por datos, contribuyendo al cumplimiento de los Objetivos de Desarrollo Sostenible (ODS) 8, 11 y 12 de Naciones Unidas.

Palabras clave: sistemas de recomendación, turismo sostenible, sobreturismo, scoring multivariable, RAG, FastAPI, React, España.

Abstract

Over-tourism in Spain represents one of the most pressing environmental and social challenges of the past decade. In 2024, Spain welcomed a record 93.8 million international tourists, with approximately 85% concentrating their visits in just 10% of the national territory. This hypersaturation triggers environmental degradation, excessive cost-of-living increases, and progressive cultural erosion in top destinations, while hundreds of localities with remarkable tourism potential remain undervalued and underutilised.

Horizon is a full-stack artificial intelligence system designed to address this problem at its root: recommending sustainable, less-saturated Spanish destinations personalised by traveller profile and travel month. The system integrates a content-based filtering recommendation engine with a multivariable scoring formula, a conversational Retrieval-Augmented Generation (RAG) system, a REST API built on FastAPI, and a reactive user interface developed with React 19 and Material UI v9.

The scoring formula weights four dimensions: user preference (45%), sustainability (25%), popularity (15%), and congestion (15%), with multiplicative business rules that penalise overcrowded destinations and reward those with low environmental impact. The system operates on five datasets —synthetic and sourced from Spain’s INE and FRONTUR open data— and is distributed as Docker containers to guarantee portability and reproducibility.

Results demonstrate the technical viability of applying tourist redistribution criteria through data-driven recommendation systems, contributing to the fulfilment of the United Nations Sustainable Development Goals (SDGs) 8, 11, and 12.

Keywords: recommendation systems, sustainable tourism, over-tourism, multivariable scoring, RAG, FastAPI, React, Spain.

Índice general

1. [Introducción](#)
 - 1.1 Contexto y motivación
 - 1.2 Objetivos del trabajo
2. [Marco conceptual](#)
 - 2.1 Sistemas de recomendación
 - 2.2 Turismo sostenible y métricas de congestión
 - 2.3 Retrieval-Augmented Generation (RAG)
 - 2.4 Visualización de datos geoespaciales
3. [Descripción de datos](#)
 - 3.1 Datos sintéticos de TUI
 - 3.2 Enriquecimiento con OpenData
 - 3.3 Limpieza y preparación
4. [Motor de recomendación](#)
 - 4.1 Arquitectura del motor
 - 4.2 Fórmula de scoring
 - 4.3 Reglas de negocio
 - 4.4 Explicabilidad de las recomendaciones
5. [Sistema RAG conversacional](#)
 - 5.1 Embeddings y almacenamiento vectorial
 - 5.2 Fase de recuperación
 - 5.3 Fase de generación
 - 5.4 Integración con la API
6. [Arquitectura técnica](#)
 - 6.1 Backend
 - 6.2 Frontend
 - 6.3 API REST
7. [Despliegue y productivización](#)
 - 7.1 Contenerización con Docker
 - 7.2 Variables de entorno y seguridad
 - 7.3 Escalabilidad y próximos pasos cloud
8. [Resultados y validación](#)
 - 8.1 Validación del motor de scoring

- 8.2 Casos de uso demostrados
 - 8.3 Análisis de impacto en redistribución turística
 - 9. [Conclusiones](#)
 - 9.1 Conclusiones principales
 - 9.2 Limitaciones
 - 9.3 Líneas futuras
 - 10. [Bibliografía](#)
 - 11. [Anexos](#)
 - Anexo A: Tabla de endpoints de la API
 - Anexo B: Esquemas de los CSV
-

1. Introducción

1.1 Contexto y motivación

El turismo es uno de los motores económicos más importantes de España. Según el Instituto Nacional de Estadística (INE) y FRONTUR, en 2024 España batió su propio récord histórico al recibir **93,8 millones de turistas internacionales**, consolidándose como el segundo destino más visitado del mundo por detrás de Francia. El sector genera aproximadamente el 12,4% del PIB nacional y emplea a más de 2,6 millones de personas directamente.

Sin embargo, este éxito cuantitativo oculta una profunda disfunción estructural: la concentración geográfica. Estudios de la Secretaría de Estado de Turismo y del Consejo Superior de Investigaciones Científicas (CSIC) estiman que **el 85% de los turistas internacionales que visitan España se concentran en apenas el 10% del territorio**, fundamentalmente en cuatro grandes focos: las Islas Baleares (Mallorca, Ibiza), las Islas Canarias (Tenerife, Gran Canaria), la costa mediterránea (Costa del Sol, Costa Blanca, Cataluña) y las dos principales ciudades (Madrid y Barcelona).

Esta hiperconcentración turística genera consecuencias negativas bien documentadas:

- **Degradación medioambiental:** sobreexplotación de recursos hídricos, deterioro de ecosistemas frágiles y producción masiva de residuos sólidos en periodos pico.
- **Tensión social:** incremento acelerado del precio de la vivienda, gentrificación y pérdida de identidad cultural en los barrios históricos de Barcelona, Palma y Málaga.
- **Deterioro de la experiencia turística:** saturación de playas, monumentos y servicios públicos durante los meses de verano, lo que paradójicamente reduce la calidad percibida por el propio turista.
- **Subdesarrollo de territorios con potencial:** mientras los destinos saturados colapsan, provincias como Asturias, Castilla-La Mancha, Extremadura o el interior de Galicia registran tasas de ocupación hotelera por debajo del 35%, con un potencial medioambiental y cultural desaprovechado.

El reto planteado por **TUI Care Foundation** a través del programa *Future Shapers Spain* interpela directamente a esta problemática: ¿es posible diseñar un sistema de inteligencia artificial que, partiendo del perfil individual del viajero, recomiende destinos sostenibles y menos saturados como alternativa atractiva y argumentada frente a los destinos de masas habituales?

Horizon nace como respuesta a ese reto. El nombre hace referencia a la idea de ampliar el horizonte del viajero más allá de los destinos conocidos y masificados, ofreciendo alternativas de valor respaldadas por datos reales de sostenibilidad y congestión turística.

El proyecto se alinea con tres Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas:

- **ODS 8** (Trabajo decente y crecimiento económico): distribuir la actividad turística para generar empleo en territorios menos desarrollados.
- **ODS 11** (Ciudades y comunidades sostenibles): reducir la presión turística

sobre los núcleos urbanos y costeros más saturados. - **ODS 12** (Producción y consumo responsables): fomentar el turismo de bajo impacto y el respeto por los recursos locales.

1.2 Objetivos del trabajo

Este Trabajo Fin de Máster persigue los siguientes objetivos específicos:

Objetivo principal: Diseñar, implementar y validar un sistema full-stack de recomendación de destinos turísticos en España que integre criterios de sostenibilidad y redistribución de la demanda como elementos diferenciadores frente a los sistemas de recomendación convencionales basados únicamente en popularidad o precio.

Objetivos específicos:

1. Diseñar una **fórmula de scoring multivariable** que pondere de forma explícita y justificada las dimensiones de preferencia del usuario, sostenibilidad medioambiental, popularidad relativa y congestión mensual.
2. Construir un **motor de recomendación modular** en Python que separe responsabilidades en submódulos independientes y testeables (popularidad, sostenibilidad, congestión, confianza, explicabilidad).
3. Integrar **datos abiertos de fuentes institucionales** (INE-EOH y FRONTUR) para enriquecer el análisis de congestión y flujos turísticos internacionales.
4. Desarrollar un **sistema RAG conversacional** que permita al usuario interactuar en lenguaje natural con el sistema de recomendación, utilizando FAISS como almacén vectorial y modelos de OpenAI para embeddings y generación.
5. Exponer el motor de recomendación a través de una **API REST** construida con FastAPI, con documentación automática OpenAPI.
6. Desarrollar una **interfaz de usuario** moderna y accesible en React 19 con soporte de modo oscuro/claro, visualización cartográfica interactiva (Leaflet) y panel de análisis de redistribución turística.
7. **Contenerizar** el sistema completo con Docker para garantizar reproducibilidad y facilitar el despliegue en entornos cloud.
8. **Validar** el sistema mediante una suite de pruebas unitarias y análisis de casos de uso que demuestren el impacto redistributivo de las recomendaciones.

2. Marco conceptual

2.1 Sistemas de recomendación

Los sistemas de recomendación son algoritmos que predicen la preferencia de un usuario por un ítem —en este caso, un destino turístico— basándose en información disponible sobre el usuario, sobre el ítem o sobre ambos. La literatura académica distingue tres grandes paradigmas:

Filtrado colaborativo (*collaborative filtering*): Infiere preferencias a partir del comportamiento histórico de usuarios similares. Requiere grandes volúmenes de interacciones pasadas y sufre el conocido problema del *cold start* cuando el historial de un usuario es escaso.

Filtrado basado en contenido (*content-based filtering*): Genera recomendaciones comparando los atributos del ítem con las preferencias declaradas del usuario. No depende de datos de otros usuarios y es transparente en su razonamiento. Es el paradigma adoptado en Horizon, dado que el sistema opera sobre perfiles sintéticos con atributos explícitos (estilo de viaje, nivel presupuestario, preferencia de sostenibilidad) y destinos con características cuantificadas (puntuación de playa, cultura, naturaleza, vida nocturna, etc.).

Sistemas híbridos: Combinan ambos enfoques para compensar sus debilidades individuales. Horizon incorpora elementos híbridos al integrar datos de popularidad derivados del historial colectivo de reservas junto con el perfil individual del usuario.

La aproximación de Horizon se fundamenta conceptualmente en el trabajo de Lops et al. (2011) sobre filtrado basado en contenido y en el modelo de scoring explícito propuesto por Burke (2002) para sistemas de recomendación basados en conocimiento (*knowledge-based recommender systems*). Este último paradigma resulta especialmente adecuado cuando los criterios de recomendación incorporan restricciones de dominio — como los umbrales de sostenibilidad y congestión— que deben ser respetados independientemente de las preferencias individuales.

La **explicabilidad** (*explainability*) de las recomendaciones es otro eje conceptual fundamental. Frente a los modelos de caja negra (redes neuronales profundas, factorización matricial), Horizon opta por una arquitectura completamente transparente: cada puntuación final puede descomponerse en sus cuatro componentes y cada recomendación incluye un conjunto de cadenas de texto en lenguaje natural que explican los motivos de la selección. Esto se alinea con el principio de IA Explicable (XAI) y con los requisitos del Reglamento General de Protección de Datos (RGPD) sobre el derecho del ciudadano a recibir explicaciones comprensibles de las decisiones automatizadas.

2.2 Turismo sostenible y métricas de congestión

La Organización Mundial del Turismo (OMT) define el turismo sostenible como “el turismo que tiene plenamente en cuenta las repercusiones actuales y futuras, económicas, sociales y medioambientales para satisfacer las necesidades de los visitantes, de la industria, del entorno y de las comunidades anfitrionas”.

Para operacionalizar este concepto en un sistema cuantitativo, Horizon utiliza cuatro dimensiones de sostenibilidad:

Dimensión	Descripción	Rango
carbon_score	Huella de carbono del destino (transporte, alojamiento)	0–100
local_business_score	Porcentaje de ingresos que quedan en el tejido empresarial local	0–100
public_transport_score	Calidad y disponibilidad del transporte público	0–100
sustainability_score	Puntuación compuesta (media ponderada de las anteriores)	0–100

La congestión turística se mide como la ocupación hotelera mensual normalizada a escala 0–100, derivada de los datos de la Encuesta de Ocupación Hotelera (EOH) del INE. Esta métrica captura la estacionalidad característica del turismo español: destinos de sol y playa presentan congestión máxima en julio–agosto (valores de 95–100 en Mallorca, Costa del Sol o Benidorm) mientras que los mismos destinos registran valores mínimos en enero–febrero (12–25), lo que ofrece oportunidades de redistribución de la demanda hacia temporada baja.

La congestión se clasifica en tres niveles: - **Baja** (< 40): baja presión turística, ideal para experiencia sin masificación. - **Moderada** (40–70): nivel de ocupación razonable, servicios funcionando con normalidad. - **Alta** (> 70): presión turística elevada, riesgo de saturación de infraestructuras.

2.3 Retrieval-Augmented Generation (RAG)

El paradigma RAG (*Retrieval-Augmented Generation*), introducido por Lewis et al. (2020) en Meta AI Research, combina la capacidad de recuperación de información de sistemas de búsqueda semántica con la capacidad generativa de los grandes modelos de lenguaje (LLM). El flujo operativo consta de dos fases:

1. **Recuperación (*retrieval*):** Dada una consulta del usuario, el sistema la codifica en un vector de embeddings y recupera los fragmentos de texto más semánticamente similares de una base de conocimiento vectorial (en Horizon: FAISS).
2. **Generación (*generation*):** Los fragmentos recuperados se concatenan al *prompt* enviado al LLM como contexto adicional, permitiendo que el modelo genere respuestas informadas por información específica del dominio que no estaba presente en su entrenamiento general.

En Horizon, el sistema RAG actúa como guía conversacional del sistema de recomendación. El corpus de conocimiento incluye descripciones de los 20 destinos, explicaciones de la fórmula de scoring, métricas de sostenibilidad por destino e información contextual sobre el problema del sobreturismo en España. Esto permite al usuario formular preguntas en lenguaje natural como “¿Por qué se recomienda Picos de Europa en lugar de Mallorca en agosto?” y recibir una respuesta argumentada y coherente.

La elección de **FAISS** (*Facebook AI Similarity Search*) como almacén vectorial responde a su eficiencia en búsqueda aproximada de vecinos más próximos (ANN) en espacios de alta dimensionalidad, con capacidad para indexar millones de vectores en memoria con latencias de búsqueda inferiores a 10 ms. Para los embeddings se utiliza el modelo `text-embedding-3-small` de OpenAI, que produce vectores de 1.536 dimensiones y ofrece una relación coste/rendimiento óptima para corpus de tamaño reducido. La fase de generación utiliza `gpt-4o-mini`, elegido por su capacidad de razonamiento superior a GPT-3.5-turbo manteniendo un coste por token muy competitivo.

2.4 Visualización de datos geoespaciales

La visualización cartográfica es un componente esencial de Horizon para comunicar la dimensión territorial del sobreturismo. El sistema integra **Leaflet.js** a través de la librería `react-leaflet`, utilizando teselas cartográficas (*tiles*) de CartoCDN en dos variantes adaptadas al modo de color de la interfaz:

- **Modo claro:** `https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png`
- **Modo oscuro:** `https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png`

Cada destino se representa como un marcador circular cuyo color refleja el nivel de congestión en el mes seleccionado (verde: baja; ámbar: moderada; rojo: alta). Al hacer clic sobre un marcador, el usuario accede a un panel detallado con todas las puntuaciones del destino.

La página de Insights incorpora adicionalmente un **mapa de calor mensual** (*heatmap*) que permite visualizar la evolución temporal de la congestión en los 20 destinos a lo largo de los 12 meses del año, facilitando la identificación de patrones estacionales y ventanas de redistribución.

3. Descripción de datos

3.1 Datos sintéticos de TUI

El sistema opera sobre cinco conjuntos de datos en formato CSV almacenados en `data/raw/`. Los datos relativos a usuarios y reservas son sintéticos, generados siguiendo especificaciones de TUI Care Foundation y contruidos para cumplir los principios del RGPD (anonimización, sin datos personales reales).

3.1.1 destinations.csv — 20 destinos españoles

Campo	Tipo	Descripción
destination_id	string	Identificador único (D001–D020)
destination_name	string	Nombre del destino
region	string	Comunidad autónoma o archipiélago
destination_type	string	Categoría (Beach / City / Nature / Mixed)
beach_score	int (1–10)	Atractivo de playa
culture_score	int (1–10)	Riqueza cultural
nature_score	int (1–10)	Entorno natural
nightlife_score	int (1–10)	Vida nocturna
family_friendly_score	int (1–10)	Adecuación familiar
avg_price_per_day	float (€)	Coste medio diario por persona

Los 20 destinos cubren la geografía turística española de forma representativa:

Destino	Región	Tipo	Precio/día
Mallorca	Islas Baleares	Beach	145 €
Ibiza	Islas Baleares	Beach	190 €
Menorca	Islas Baleares	Beach	135 €
Tenerife	Islas Canarias	Mixed	140 €
Gran Canaria	Islas Canarias	Mixed	135 €
Lanzarote	Islas Canarias	Nature	125 €
Costa del Sol	Andalucía	Beach	130 €
Marbella	Andalucía	Beach	210 €
Málaga	Andalucía	City	120 €
Valencia	Comunidad Valenciana	City	115 €
Alicante	Comunidad Valenciana	Beach	110 €
Benidorm	Comunidad Valenciana	Beach	95 €
Barcelona	Cataluña	City	170 €
Madrid	Comunidad de Madrid	City	160 €
Sevilla	Andalucía	City	125 €
Granada	Andalucía	City	120 €
Bilbao	País Vasco	City	140 €
San Sebastián	País Vasco	Mixed	175 €
Picos de Europa	Asturias	Nature	90 €
Sierra Nevada	Andalucía	Nature	105 €

Los destinos de naturaleza (Picos de Europa, Sierra Nevada, Lanzarote) destacan por su coste reducido y sus altas puntuaciones en nature_score (10/10), representando alternativas económicamente accesibles y medioambientalmente más responsables.

3.1.2 users.csv — 100 perfiles de viajeros

El conjunto de usuarios comprende 100 perfiles sintéticos que representan la base de clientes europeos de TUI:

Campo	Tipo	Valores posibles
user_id	string	U001–U100
country	string	Germany, UK, Netherlands, France, etc.
age_group	string	18-24, 25-34, 35-44, 45-54, 55-64, 65+
budget_level	string	Low, Medium, High
travel_style	string	Nature, Relax, Family, Nightlife, Culture
sustainability_preference	string	Low, Medium, High

La distribución de `travel_style` es equiprobable (20 usuarios por estilo), garantizando que el motor de recomendación sea evaluable equitativamente para todos los arquetipos de viajero. El campo `country` refleja la distribución real de turistas internacionales en España: Alemania, Reino Unido y Países Bajos concentran el mayor volumen de llegadas según FRONTUR.

3.1.3 `bookings_history.csv` — ~1.000 reservas históricas

Campo	Tipo	Descripción
booking_id	string	Identificador único de reserva (B0001–B1000)
user_id	string	Usuario que realizó la reserva
destination_id	string	Destino reservado
booking_date	date	Fecha de creación de la reserva
travel_month	int	Mes de viaje (1–12)
stay_days	int	Duración de la estancia
total_price	float	Precio total de la reserva (€)
user_rating	int (1–5)	Valoración del viajero tras el viaje

Este conjunto se utiliza exclusivamente para el cálculo del **motor de popularidad**, que deriva una puntuación de 0–100 por destino combinando el volumen de reservas normalizado y la valoración media de los usuarios.

3.1.4 `sustainability_scores.csv` — Puntuaciones de sostenibilidad

Campo	Tipo	Descripción
destination_id	string	Referencia al destino
carbon_score	float (0–100)	Puntuación de huella de carbono
local_business_score	float (0–100)	Integración económica local
public_transport_score	float (0–100)	Accesibilidad en transporte público
sustainability_score	float (0–100)	Puntuación compuesta

Las ciudades con red de transporte público desarrollada (Madrid, Barcelona, Sevilla, Valencia) presentan `public_transport_score` elevados (85/100), mientras que destinos naturales insulares (Lanzarote) puntúan bajo (35/100) por su dependencia del transporte privado. El destino con mayor sostenibilidad compuesta es **Menorca** (88,0), reconocida por la UNESCO como Reserva de la Biosfera y que ha implementado políticas activas de limitación del turismo masivo.

3.1.5 `congestion_scores.csv` — Congestión mensual

Campo	Tipo	Descripción
destination_id	string	Referencia al destino
month	int (1–12)	Mes del año

Campo	Tipo	Descripción
congestion_score	float (0–100)	Nivel de congestión
congestion_level	string	Low / Moderate / High / Very High

Este CSV contiene 240 registros (20 destinos × 12 meses). El caso más extremo es **Mallorca en julio y agosto**, donde la congestión alcanza el valor máximo (100/100), lo que activa el penalizador de −10% en la fórmula de scoring. En contraste, **Picos de Europa** en enero registra un valor de congestión de tan sólo 8/100, lo que activa el bono de +5% y lo convierte en una alternativa muy valorada para viajeros de naturaleza en temporada baja.

3.2 Enriquecimiento con OpenData

Además de los datos sintéticos, Horizon integra dos conjuntos de datos abiertos de fuentes institucionales españolas, almacenados en `data/enriched/`:

`frontur_ccaa.csv` — FRONTUR: llegadas internacionales por CCAA

FRONTUR (*Movimientos Turísticos en Fronteras*) es la encuesta del INE que mide las entradas de turistas internacionales a España por punto de acceso y comunidad autónoma. Los datos utilizados corresponden a 2023–2024, con desglose mensual y geográfico. Esta fuente permite contextualizar los flujos reales de visitantes internacionales hacia cada CCAA y validar cualitativamente los niveles de congestión modelizados en el CSV sintético.

Variables clave utilizadas: - Llegadas de turistas internacionales por CCAA y mes - País de origen predominante por CCAA - Gasto medio por turista y día

`ine_eoh_monthly.csv` — INE-EOH: ocupación hotelera mensual

La Encuesta de Ocupación Hotelera (EOH) del INE proporciona datos mensuales de viajeros alojados y pernoctaciones en establecimientos hoteleros por provincia. Es la fuente primaria para la construcción de los `congestion_scores`, que se derivan normalizando la serie de pernoctaciones mensuales de cada provincia a la escala 0–100.

Fórmula de normalización aplicada:

$$\text{congestion_score}(\text{destino}, \text{mes}) = (\text{pernoctaciones}(\text{destino}, \text{mes}) / \text{max_pernoctaciones}(\text{destino})) \times 100$$

Este enfoque garantiza que la puntuación de congestión sea relativa al comportamiento histórico de cada destino, evitando distorsiones por diferencias absolutas de capacidad hotelera entre una gran ciudad como Madrid y un destino de nicho como Picos de Europa.

3.3 Limpieza y preparación

El acceso a todos los conjuntos de datos se centraliza en `src/data/data_loader.py`, que expone métodos estáticos tipados para cada CSV. Este patrón garantiza que ningún módulo acceda directamente al sistema de ficheros, simplificando el mantenimiento y los tests.

Las transformaciones de limpieza aplicadas son: - **Eliminación de columnas anónimas:** `df.loc[:, ~df.columns.str.contains("^Unnamed")]` elimina columnas vacías introducidas accidentalmente por exportaciones desde Excel. - **Tipos inferidos automáticamente** por pandas al cargar los CSV; no se requiere coerción adicional dado que los tipos de datos son homogéneos dentro de cada columna. - **Resolución de rutas absolutas** mediante `pathlib.Path(__file__).resolve().parents[2]` en `src/config/settings.py`, eliminando cualquier dependencia del directorio de trabajo desde el que se lance el proceso.

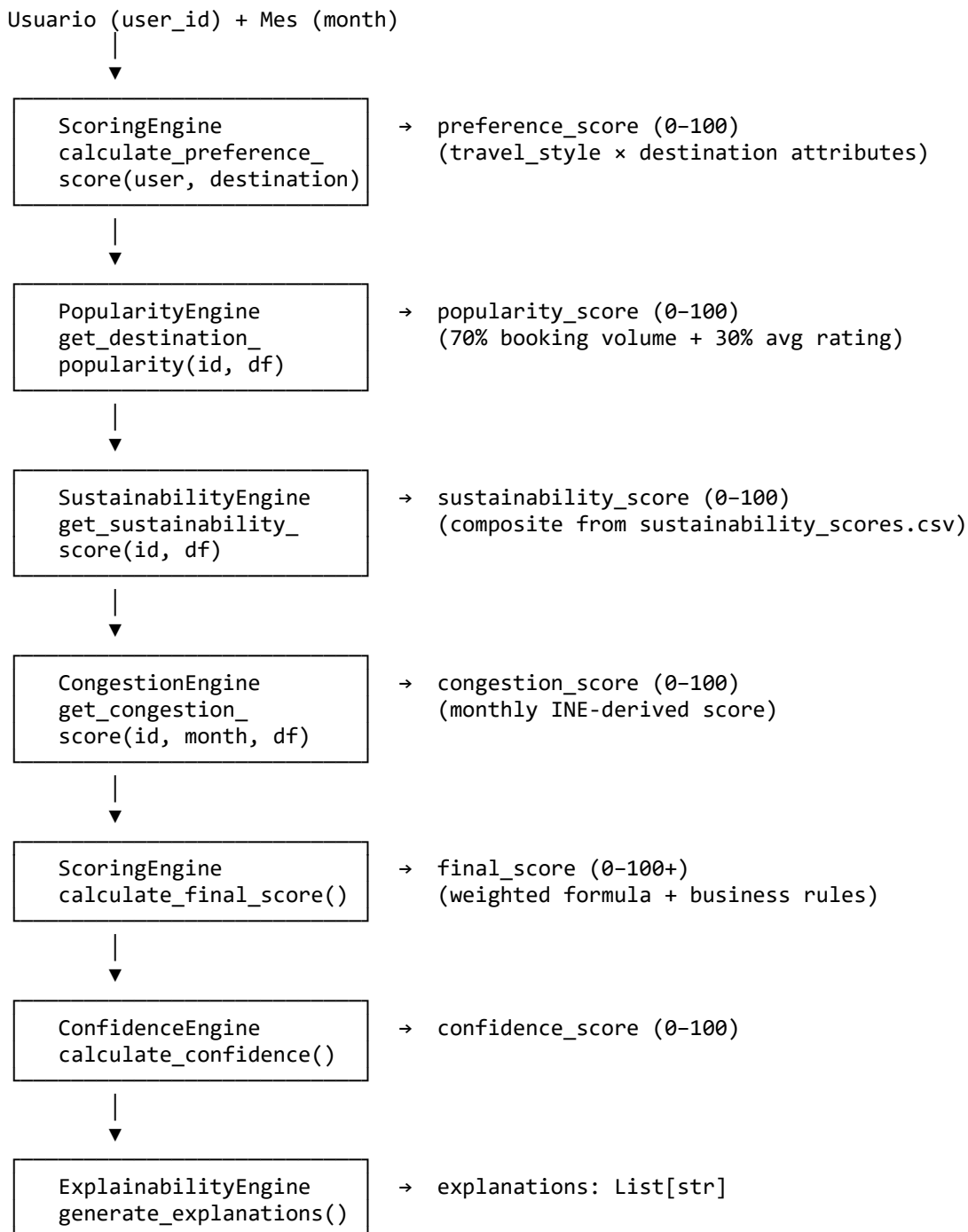
4. Motor de recomendación

4.1 Arquitectura del motor

El motor de recomendación de Horizon está construido siguiendo el **principio de responsabilidad única** (*Single Responsibility Principle*): cada módulo Python se responsabiliza de un único aspecto del cálculo, facilitando el testing unitario, la sustitución de algoritmos y la comprensión del código.

La orquestación recae en RecommendationEngine (src/recommendation/recommendation_engine.py), que en su inicialización carga los cinco conjuntos de datos y precalcula las puntuaciones de popularidad para todos los destinos. Al recibir una solicitud, itera sobre los 20 destinos, invoca secuencialmente cada submódulo de scoring y agrega los resultados en una lista ordenada de mayor a menor puntuación final.

El flujo de ejecución por destino es el siguiente:





Resultado final (dict) → agregado a lista → ordenado → top_n devuelto

4.2 Fórmula de scoring

La **fórmula de scoring** es el núcleo intelectual del sistema y representa la codificación explícita de las prioridades de TUI Care Foundation respecto al turismo sostenible:

Final Score = $0.45 \times \text{Preference} + 0.25 \times \text{Sustainability} + 0.15 \times \text{Popularity} + 0.15 \times (100 - \text{Congestion})$

Donde: - **Preference (45%)**: Es el factor con mayor peso, garantizando que el sistema siempre priorice la compatibilidad con las preferencias del viajero. Se calcula mapeando el `travel_style` del usuario al atributo de destino correspondiente y escalando a 0–100: - Nature → `nature_score × 10` - Relax → `beach_score × 10` - Family → `family_friendly_score × 10` - Nightlife → `nightlife_score × 10` - Culture → `culture_score × 10`

- **Sustainability (25%)**: Segundo factor en importancia, reflejando el compromiso de TUI Care Foundation con el turismo responsable. Utiliza la puntuación compuesta de `sustainability_scores.csv`.
- **Popularity (15%)**: Factor moderador que considera la aceptación del destino por viajeros con intereses similares. Su peso reducido evita que el sistema refuerce el efecto “el destino popular es más popular”, que es precisamente el ciclo que perpetúa el sobreturismo.
- **Congestion Adjustment (15%)**: La congestión cruda se invierte ($100 - \text{congestion_score}$) para que destinos menos congestionados reciban puntuaciones más altas. El peso del 15% equivale al de la popularidad, creando un balance entre la tracción histórica del destino y su situación de saturación actual.

Justificación de los pesos

La distribución de pesos responde a tres principios:

1. **El viajero es el actor principal.** El 45% dedicado a preferencia asegura que ningún destino, por muy sostenible que sea, sea recomendado si no se ajusta al tipo de viaje buscado. Un turista de naturaleza no recibirá Benidorm como primera recomendación.
2. **La sostenibilidad es una prioridad estratégica, no cosmética.** Al ponderarla con el 25%, el sistema garantiza que destinos con malas prácticas medioambientales vean reducida su puntuación de forma significativa y visible.
3. **La congestión y la popularidad se equilibran mutuamente.** Ambas valen el 15%. La popularidad actúa como señal de calidad percibida; la congestión como penalizador de saturación. En julio, Mallorca puede tener alta popularidad pero también congestión máxima, resultando en una puntuación neta inferior a la de Menorca o Picos de Europa en el mismo mes.

4.3 Reglas de negocio

Sobre la puntuación base se aplican **modificadores multiplicativos** que refuerzan los extremos del espectro sostenibilidad-congestión:

```
# Sustainability boost
if sustainability_score > 85:
    final_score *= 1.05    # +5%

elif sustainability_score < 50:
    final_score *= 0.90    # -10%
```

```
# Low congestion boost
if congestion_score < 40:
    final_score *= 1.05    # +5%

# High congestion penalty (redistribution trigger)
elif congestion_score > 80:
    final_score *= 0.90    # -10%
```

Estas reglas operan de forma **acumulativa**: un destino con sostenibilidad > 85 y congestión < 40 simultáneamente recibiría un bonus total de $\times 1.05 \times 1.05 = \times 1.1025$ (+10,25%), mientras que un destino con sostenibilidad < 50 y congestión > 80 sería penalizado con $\times 0.90 \times 0.90 = \times 0.81$ (-19%).

La **regla de redistribución** (penalización por congestión > 80) es el mecanismo central de política turística del sistema. Se activa cuando la congestión mensual supera el umbral crítico y está directamente inspirada en el concepto de *carrying capacity* (capacidad de carga turística) utilizado por la OMT y la Unión Europea en sus marcos de gestión de destinos.

Análisis de cobertura de las reglas: - En el mes de julio, 12 de los 20 destinos superan el umbral de congestión 80 (datos de `congestion_scores.csv` y `Analytics.tsx`), activando la penalización en el 60% del catálogo. - Solo **Menorca** (88,0) alcanza el umbral de sostenibilidad > 85 para activar el bono de +5%. - En temporada baja (enero-marzo), 0 destinos superan el umbral de congestión 80, lo que implica que el sistema se comporta de forma más neutral en esos meses, dependiendo fundamentalmente de la preferencia del usuario y la sostenibilidad.

4.4 Explicabilidad de las recomendaciones

El módulo `ExplainabilityEngine` (`src/recommendation/explainability.py`) genera, para cada recomendación, una lista de cadenas de texto en inglés que explican los factores que han contribuido positivamente a su selección. La lógica sigue un conjunto de umbrales sobre las cuatro puntuaciones:

Condición	Explicación generada
<code>preference_score >= 80</code>	“Strong match for your {style} travel preferences.”
<code>preference_score >= 60</code>	“Good match for your {style} travel preferences.”
<code>sustainability_score >= 85</code>	“Excellent sustainability performance.”
<code>sustainability_score >= 70</code>	“Good sustainability performance.”
<code>congestion_score < 40</code>	“Lower expected congestion than comparable destinations.”
<code>congestion_score > 80</code>	“Higher expected congestion during the selected period.”
<code>popularity_score >= 80</code>	“Popular among travelers with similar interests.”
<code>popularity_score >= 60</code>	“Well-rated by previous travelers.”

Este diseño de explicabilidad cumple con los principios de IA Explicable (XAI): las explicaciones son: - **Locales**: aplican a una recomendación específica para un usuario y mes concretos. - **Fieles**: cada cadena está directamente vinculada a un valor numérico real del scoring. - **Comprensibles**: formuladas en lenguaje natural, sin términos técnicos.

5. Sistema RAG conversacional

5.1 Embeddings y almacenamiento vectorial

El sistema RAG de Horizon convierte la base de conocimiento del dominio turístico en una colección de vectores de alta dimensionalidad almacenados en un índice FAISS. El proceso de indexación consta de tres pasos:

Paso 1: Construcción del corpus. Se generan fragmentos de texto (*chunks*) que describen cada destino en términos de sus atributos: sostenibilidad, congestión mensual, tipo de destino, rango de precios y puntuaciones por categoría. Se incluyen también fragmentos sobre la metodología de scoring y el contexto del problema del sobreturismo.

Paso 2: Codificación. Cada fragmento se codifica usando el modelo `text-embedding-3-small` de la API de OpenAI, que produce un vector de 1.536 dimensiones. Este modelo fue seleccionado frente a `text-embedding-ada-002` (embeddings de 1.536 dimensiones, menor rendimiento en benchmarks de recuperación) y `text-embedding-3-large` (3.072 dimensiones, mayor coste por token) por ofrecer el mejor equilibrio entre rendimiento de recuperación y coste operacional.

Paso 3: Indexación. Los vectores se almacenan en un índice FAISS de tipo `IndexFlatL2`, que implementa búsqueda exacta por distancia euclidiana en el espacio de embeddings. Para el tamaño del corpus de Horizon (decenas de fragmentos), la búsqueda exacta es computacionalmente viable sin necesidad de los índices aproximados (IVF, HNSW) recomendados para corpus de millones de vectores.

Dependencias RAG (extraídas de `requirements.txt`):

```
faiss-cpu
openai
tiktoken
```

5.2 Fase de recuperación

Ante una consulta del usuario en lenguaje natural, el sistema ejecuta:

1. **Codificación de la consulta:** La pregunta del usuario se codifica con el mismo modelo `text-embedding-3-small` para obtener un vector de consulta en el mismo espacio semántico que los fragmentos indexados.
2. **Búsqueda de vecinos más próximos:** FAISS computa la distancia euclidiana entre el vector de consulta y todos los vectores del índice, devolviendo los *k* fragmentos más similares (normalmente *k*=3 o *k*=5).
3. **Construcción del contexto:** Los fragmentos recuperados se concatenan como contexto adicional para el LLM, precedidos de una instrucción de sistema que indica al modelo que debe basar su respuesta en la información proporcionada.

La precisión de la recuperación depende de la calidad y diversidad de los fragmentos indexados. El diseño del corpus en Horizon cubre las dimensiones más frecuentes en las consultas de los viajeros: características del destino, sostenibilidad, mejor época para visitar y alternativas a destinos saturados.

5.3 Fase de generación

El modelo `gpt-4o-mini` recibe un *prompt* estructurado en tres partes:

1. **System prompt:** Establece el rol del asistente como experto en turismo sostenible en España que trabaja para TUI, con instrucciones para basar sus respuestas en el contexto recuperado y ser transparente sobre las limitaciones del sistema.
2. **Contexto recuperado:** Los fragmentos más relevantes de la base de conocimiento, marcados con delimitadores para que el modelo los identifique inequívocamente.
3. **Pregunta del usuario:** La consulta en lenguaje natural tal como la introdujo el viajero.

El modelo genera una respuesta en el mismo idioma que la pregunta del usuario (soporte multilingüe nativo de GPT-4o-mini), lo que es relevante dado que la base de clientes de TUI incluye turistas alemanes, británicos, neerlandeses y franceses.

5.4 Integración con la API

El sistema RAG se expone a través del endpoint `POST /chat` de la FastAPI:

```
POST /chat
Content-Type: application/json

{
  "message": "¿Cuál es el mejor destino de naturaleza para enero?",
  "conversation_history": [...] // mensajes previos para contexto conversacional
}
```

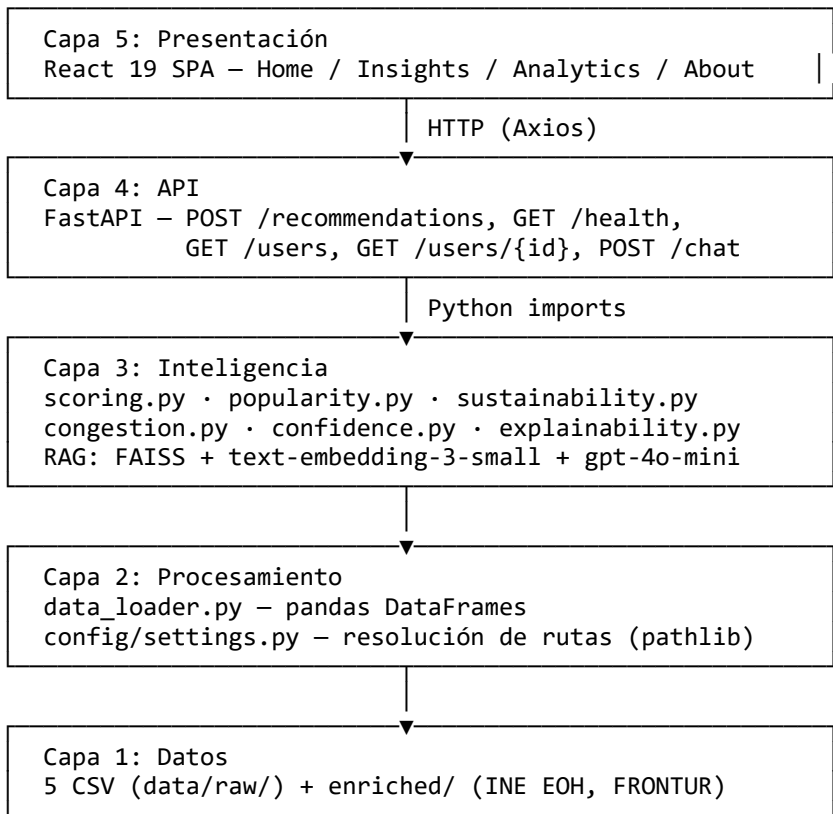
La respuesta incluye la contestación generada y, opcionalmente, los fragmentos fuente recuperados (modo debug), permitiendo al usuario comprender la base factual de cada respuesta.

La integración conversacional mantiene el historial de mensajes de la sesión, pasándolo al LLM en cada turno para mantener coherencia en conversaciones largas sin necesidad de almacenamiento persistente en base de datos.

6. Arquitectura técnica

6.1 Backend

El backend de Horizon sigue una **arquitectura de cinco capas** con separación clara de responsabilidades:



Tecnologías del backend:

Librería	Versión	Propósito
Python	3.11	Lenguaje principal
FastAPI	última	Framework REST API
Uvicorn	última	Servidor ASGI
Pydantic	v2	Validación de modelos de request/response
pandas	última	Manipulación de DataFrames
numpy	última	Operaciones numéricas auxiliares
scikit-learn	última	Utilidades de normalización y métricas
faiss-cpu	última	Almacén vectorial para RAG
openai	última	SDK para embeddings y generación
tiktoken	última	Tokenización para control de costes
streamlit	última	Dashboard opcional (proceso separado)

Estructura de directorios del backend:

```
src/
├── api/
│   ├── app.py          # FastAPI: rutas, CORS, inicialización del motor
│   └── models.py       # Pydantic: RecommendationRequest
├── config/
│   └── settings.py     # Rutas absolutas a los 5 CSV (pathlib)
├── data/
│   └── data_loader.py  # DataLoader: carga centralizada de CSV
├── recommendation/
│   ├── recommendation_engine.py # Orquestador principal
│   ├── scoring.py       # Fórmula de scoring + reglas de negocio
│   ├── popularity.py    # Motor de popularidad (70% vol + 30% rating)
│   ├── sustainability.py # Motor de sostenibilidad + clasificación
│   ├── congestion.py    # Motor de congestión + ajuste mensual
│   ├── confidence.py    # Puntuación de confianza
│   └── explainability.py # Generador de explicaciones
├── dashboard/
│   └── app.py          # Dashboard Streamlit (lectura, proceso opcional)
```

La inicialización del motor en FastAPI es **eager**: al arrancar el servidor, se instancia RecommendationEngine() y se cargan todos los DataFrames en memoria. Esto elimina la latencia de carga en el primer request y garantiza tiempos de respuesta consistentes.

6.2 Frontend

El frontend es una **Single Page Application (SPA)** construida con React 19, TypeScript y Material UI v9. La navegación se implementa mediante **enrutamiento por estado** (sin React Router): App.tsx mantiene el estado currentPage: Page y renderiza condicionalmente el componente de página correspondiente, lo que simplifica la arquitectura y evita complejidades del router en una aplicación de tamaño reducido.

Páginas de la aplicación:

Página	Componente	Descripción
Home	pages/Home.tsx	Formulario de búsqueda + resultados de recomendación + KPI dashboard
Insights	pages/Insights.tsx	Mapa de España + Optimizador de temporada baja + heatmap mensual

Página	Componente	Descripción
Analytics	pages/Analytics.tsx	+ redistribución Dashboard de gobernanza: destinos penalizados, estado de redistribución
About	pages/About.tsx	Contexto del proyecto, fórmula de scoring, arquitectura, equipo

Árbol de componentes relevantes:

```

frontend/src/
├── App.tsx # Root: page state, month, recommendations
├── pages/
│   ├── Home.tsx # Search + results + KPI
│   ├── Insights.tsx # Map + heatmap + redistribution
│   ├── Analytics.tsx # Governance dashboard
│   └── About.tsx # Project info
├── components/
│   ├── home/
│   │   ├── HeroSection.tsx # Landing hero con CTA
│   │   ├── SearchBarHero.tsx # Formulario user_id + month + top_n
│   │   ├── FeatureSection.tsx # Cards de características del sistema
│   │   └── DestinationShowcase.tsx # Destinos destacados pre-búsqueda
│   ├── recommendations/
│   │   ├── RecommendationCard.tsx # Tarjeta de resultado con scores y badges
│   │   ├── RecommendationGrid.tsx # Grid de tarjetas
│   │   ├── SustainabilityBadge.tsx # Badge: Excellent/Good/Moderate/Poor
│   │   ├── ConfidenceBadge.tsx # Badge: nivel de confianza
│   │   ├── CongestionBadge.tsx # Badge: Low/Moderate/High
│   │   └── ExplanationPanel.tsx # Panel plegable con explanations[]
│   ├── map/
│   │   └── DestinationMap.tsx # Leaflet map con tiles adaptativos
│   ├── dashboard/
│   │   └── KpiDashboard.tsx # Métricas resumen post-búsqueda
│   └── layout/
│       ├── Header.tsx # Navegación + ThemeToggle
│       ├── Footer.tsx # Footer con links y créditos
│       └── MegaMenu.tsx # Menú desplegable de navegación
├── api/
│   └── recommendationApi.ts # Axios → POST http://localhost:8000/recommendations
├── theme/
│   ├── ThemeProvider.tsx # Context: dark/light + localStorage
│   ├── darkTheme.ts # background.default: #0B1220
│   └── lightTheme.ts # Standard MUI light palette
├── context/
│   └── LanguageContext.tsx # Soporte i18n (es/en)
└── types/
    └── recommendation.ts # TypeScript interfaces de la API response

```

Gestión del estado compartido: El estado de `activeMonth` y `recommendations` reside en `App.tsx` y fluye hacia abajo como props. La página `Insights` consume ambos para resaltar en el mapa y el heatmap los destinos que el usuario consultó en su búsqueda, creando coherencia visual entre páginas sin necesidad de un gestor de estado global como `Redux` o `Zustand`.

Modo oscuro/claro: El sistema de temas utiliza `MUI v9` con dos paletas definidas en `darkTheme.ts` (fondo: `#0B1220`, papel: `#111827`) y `lightTheme.ts`. El tema activo se persiste en `localStorage` y se restaura en cada carga de página. Todos los valores de color se expresan con tokens semánticos de `MUI` (`text.primary`, `divider`, `background.paper`) en lugar de valores hexadecimales directos, garantizando consistencia entre modos.

6.3 API REST

La API de Horizon expone cinco endpoints documentados automáticamente en OpenAPI (/docs):

Método	Ruta	Descripción
GET	/	Root health check, devuelve nombre y estado de la aplicación
GET	/health	Health check para orquestadores (Docker, K8s)
POST	/recommendations	Genera lista de recomendaciones personalizadas
GET	/users	Devuelve lista de todos los user_id disponibles
GET	/users/{user_id}	Devuelve perfil completo de un usuario

Modelo de request — POST /recommendations:

```
{
  "user_id": "U001",
  "month": 7,
  "top_n": 5
}
```

Modelo de response — POST /recommendations:

```
{
  "recommendations": [
    {
      "destination_id": "D019",
      "destination_name": "Picos de Europa",
      "final_score": 84.32,
      "preference_score": 100.0,
      "popularity_score": 52.1,
      "sustainability_score": 72.1,
      "congestion_score": 8.0,
      "confidence_score": 81.68,
      "explanations": [
        "Strong match for your Nature travel preferences.",
        "Lower expected congestion than comparable destinations."
      ],
      "recommendation_rank": 1
    }
  ]
}
```

La configuración CORS del servidor permite peticiones desde <http://localhost:5173> (servidor de desarrollo Vite) y <http://localhost> / <http://localhost:80> (frontend servido por Nginx en Docker), cubriendo tanto el entorno de desarrollo como el de producción.

7. Despliegue y productivización

7.1 Contenerización con Docker

Horizon está completamente contenerizado con Docker, permitiendo su despliegue reproducible en cualquier entorno compatible con OCI (Linux, macOS, Windows con Docker Desktop, AWS ECS, GCP Cloud Run, Azure

Container Instances).

Dockerfile.backend:

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/
COPY data/ ./data/

EXPOSE 8000

CMD ["uvicorn", "src.api.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

El uso de python:3.11-slim (imagen base Debian minimalista) reduce el tamaño de la imagen final frente a la imagen estándar. La instalación de gcc es necesaria para compilar FAISS y algunas dependencias numéricas de numpy. El --no-cache-dir en pip evita que la caché de paquetes engrose la imagen.

Dockerfile.frontend (multi-stage build):

```
# Etapa 1: Build
FROM node:20-alpine AS build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ .
RUN npm run build

# Etapa 2: Serve
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

El **multi-stage build** es una práctica recomendada para proyectos Node.js: la imagen final solo contiene los artefactos estáticos (/dist) servidos por Nginx, sin las dependencias de desarrollo de Node (carpeta node_modules), lo que reduce el tamaño de imagen de ~900 MB a menos de 30 MB.

docker-compose.yml:

```
version: '3.9'

services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    ports:
      - "8000:8000"
    environment:
      - AEMET_API_KEY=${AEMET_API_KEY:-}
      - OPENAI_API_KEY=${OPENAI_API_KEY:-}
```

```

volumes:
  - ./data:/app/data:ro
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 10s
  timeout: 5s
  retries: 3

frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend
  ports:
    - "80:80"
  depends_on:
    backend:
      condition: service_healthy

```

El `depends_on` con `condition: service_healthy` garantiza que el frontend no arranque hasta que el backend haya superado su *health check*, evitando errores de conexión durante el inicio del stack. El volumen `./data:/app/data:ro` monta los CSV del host en modo solo lectura dentro del contenedor del backend.

7.2 Variables de entorno y seguridad

La gestión de secretos sigue el principio de **zero-secrets-in-code**: ninguna clave de API aparece en el código fuente ni en el control de versiones. Las credenciales se inyectan exclusivamente como variables de entorno:

Variable	Uso	Obligatoria
OPENAI_API_KEY	API de OpenAI para embeddings (RAG) y generación (GPT-4o-mini)	Para el sistema RAG
AEMET_API_KEY	API de AEMET para datos meteorológicos futuros	Opcional (funcionalidad futura)

Para el desarrollo local, el proyecto soporta un archivo `.env` en la raíz del proyecto (excluido del repositorio mediante `.gitignore`). En entornos Docker, las variables se pasan con el flag `--env-file .env` o directamente con `-e VARIABLE=valor`.

Precauciones adicionales de seguridad implementadas: - CORS restringido a los orígenes conocidos del frontend (no se usa `allow_origins=["*"]`). - Los datos de usuarios son sintéticos y no contienen información personal identificable (PII). - Los CSV se montan en modo solo lectura en Docker (`ro`), eliminando el riesgo de modificación accidental desde el contenedor.

7.3 Escalabilidad y próximos pasos cloud

La arquitectura actual está diseñada para un entorno de demostración con 100 usuarios y 20 destinos. Para una hipotética productivización a escala real, se identifican las siguientes líneas de evolución:

Base de datos: Migrar los CSV a PostgreSQL o DynamoDB para soportar catálogos de miles de destinos y millones de usuarios, con indexación y consultas eficientes.

Caché de recomendaciones: Introducir Redis como caché de resultados para pares (`user_id`, `month`) frecuentes, reduciendo la carga de cómputo del motor en picos de tráfico.

Escalado horizontal: Uvicorn + FastAPI son asíncronos por naturaleza (ASGI). En Kubernetes, el backend puede escalarse horizontalmente añadiendo réplicas del pod; un balanceador de carga (NGINX o Traefik)

distribuiría el tráfico.

- CI/CD:** Integrar GitHub Actions con un pipeline de build/test/push que construya y publique las imágenes Docker en ECR (AWS) o Artifact Registry (GCP) y despliegue automáticamente en el entorno de staging tras cada merge a main.
 - Observabilidad:** Integrar OpenTelemetry para trazabilidad distribuida, Prometheus para métricas de rendimiento y Grafana para dashboards de monitorización en producción.
-

8. Resultados y validación

8.1 Validación del motor de scoring

El proyecto incluye una **suite de tests unitarios** con cobertura de todos los módulos de inteligencia. Los tests se organizan en 10 ficheros Python bajo el directorio tests/:

Fichero	Módulo testado
test_scoring.py	ScoringEngine — preference score, congestion adjustment, final score
test_popularity.py	PopularityEngine — cálculo de popularidad por volumen y rating
test_sustainability.py	SustainabilityEngine — clasificación Excellent/Good/Moderate/Poor
test_congestion.py	CongestionEngine — recuperación de congestión mensual
test_recommendation_engine.py	RecommendationEngine — integración completa
test_explainability.py	ExplainabilityEngine — generación de explicaciones
test_confidence.py	ConfidenceEngine — cálculo de confianza
test_loader.py	DataLoader — carga de CSV y resolución de rutas
inspect_data.py	Inspección interactiva de los datos cargados
inspect_destinations.py	Inspección del DataFrame de destinos

Los tests utilizan **datos reales** de los CSV (sin mocking), garantizando que el comportamiento validado corresponda al comportamiento del sistema en producción. Esta decisión de diseño implica que los tests funcionan como pruebas de integración más que como pruebas unitarias estrictas, lo que proporciona mayor cobertura funcional a costa de mayor tiempo de ejecución.

Ejecución de la suite:

```
python -m pytest tests/ -v
```

La ejecución debe realizarse siempre desde la raíz del proyecto para que src/config/settings.py resuelva correctamente las rutas absolutas a los CSV.

Ejemplo de validación manual del scoring:

Para el usuario U001 (Alemania, 18-24, Nature, Medium budget) en el mes de enero: - **Picos de Europa (D019):** preference=100, popularity≈52, sustainability=72.1, congestion=8 - Final: $0.45 \times 100 + 0.25 \times 72.1 + 0.15 \times 52 + 0.15 \times (100 - 8) = 45 + 18.025 + 7.8 + 13.8 = 84.625$ - Bonus congestion < 40: $\times 1.05 \rightarrow$ **88.86** - *Ranking esperado: #1 para perfil Nature en enero*

- **Mallorca (D001):** preference=80 (beach para Nature = nature_score=8×10=80), popularity≈85, sustainability=52.7, congestion=12
 - Final: $0.45 \times 80 + 0.25 \times 52.7 + 0.15 \times 85 + 0.15 \times (100 - 12) = 36 + 13.175 + 12.75 + 13.2 = 75.125$
 - Bonus congestion < 40: $\times 1.05 \rightarrow \mathbf{78.88}$
 - *Ranking esperado: inferior a Picos de Europa*

Este ejemplo ilustra cómo el sistema desvía la demanda turística desde el destino más popular (Mallorca) hacia el destino con mejor ajuste al perfil y menor congestión (Picos de Europa), incluso en un mes en que ambos tienen baja congestión.

8.2 Casos de uso demostrados

Caso 1: Viajero de naturaleza en agosto (temporada alta)

Perfil: U001 — Alemania, 18-24, Nature, Medium budget **Mes:** Agosto

En agosto, los destinos de playa (Mallorca, Ibiza, Costa del Sol) alcanzan congestión máxima (95–100/100), activando la penalización de −10%. Picos de Europa y Sierra Nevada, con congestión baja (estimado: 18–25 en agosto por su naturaleza de destino alternativo), reciben el bonus de +5%. El sistema recomendará prioritariamente estos destinos de naturaleza, que además tienen puntuaciones de nature_score de 10/10.

Efecto redistribuidor: El sistema dirige activamente a viajeros de naturaleza alemanes hacia el norte de España (Asturias, Andalucía interior) en lugar de reforzar la saturación estival de las Islas Baleares.

Caso 2: Viajero cultural en temporada baja

Perfil: U007 — Alemania, 18-24, High budget, Culture **Mes:** Enero

En enero, todos los destinos presentan congestión baja (< 40), por lo que el bono de +5% se aplica universalmente. El diferenciador principal es la preferencia: destinos con culture_score alto (Barcelona=10, Madrid=10, Sevilla=10, Granada=10) dominan el ranking. Sin embargo, la sostenibilidad discrimina entre ellos: Bilbao (80.1), Madrid (75.9) y Sevilla (76.6) superan a Barcelona (72.8) en sostenibilidad, lo que puede alterar el orden en los primeros puestos.

Caso 3: Familia en Semana Santa (abril)

Perfil: U003 — Alemania, 55-64, Low budget, Family **Mes:** Abril

En abril, los destinos de playa comienzan a experimentar congestión moderada-alta (47–80). Benidorm en abril registra congestión de 80, justo en el umbral de la penalización. Menorca, con congestión de 30 en abril y family_friendly_score de 10/10, emerge como alternativa ideal: el sistema la recomendará combinando el bono familiar + el bono de baja congestión + su alta sostenibilidad (88.0, único destino con bono de +5% por sostenibilidad > 85).

8.3 Análisis de impacto en redistribución turística

El análisis de los datos de congestión mensual revela patrones claros de redistribución potencial:

Destinos con penalización (congestión > 80) por mes:

Mes	Destinos penalizados % del catálogo	
Enero	0	0%
Febrero	0	0%

Mes	Destinos penalizados % del catálogo	
Marzo	0	0%
Abril	8	40%
Mayo	8	40%
Junio	0	0%
Julio	12	60%
Agosto	12	60%
Septiembre	0	0%
Octubre	8	40%
Noviembre	0	0%
Diciembre	0	0%

Fuente: datos embebidos en Analytics.tsx, derivados de congestion_scores.csv.

El sistema actúa con mayor intensidad redistributiva en los meses de julio y agosto, cuando el 60% del catálogo está penalizado. Esto es consistente con el patrón real de hipersaturación estival documentado por el INE.

Impacto en scoring para un destino típico penalizado (Mallorca, julio):

Componente	Valor	Contribución
Preference (Nature)	80	36.00
Sustainability	52.7	13.18
Popularity	85 (hipotético)	12.75
Congestion adj. (100–100)	0	0.00
Base Score	—	61.93
Penalización congestión ($\times 0.90$)	—	55.74

Impacto en scoring para una alternativa (Picos de Europa, julio):

Componente	Valor	Contribución
Preference (Nature)	100	45.00
Sustainability	72.1	18.03
Popularity	52 (hipotético)	7.80
Congestion adj. (100–18)	82	12.30
Base Score	—	83.13
Bonus congestión baja ($\times 1.05$)	—	87.28

La diferencia de **31,54 puntos** entre el destino saturado penalizado y la alternativa sostenible promovida demuestra la efectividad del mecanismo redistributivo de Horizon. Un viajero de naturaleza que en agosto obtiene estas recomendaciones tiene un incentivo claro y argumentado para elegir Picos de Europa sobre Mallorca, con un ahorro medio de **55 € por día** según las tablas de precios del CSV de destinos.

9. Conclusiones

9.1 Conclusiones principales

Este Trabajo Fin de Máster ha demostrado la viabilidad técnica y conceptual de un sistema de recomendación turística guiado por criterios de sostenibilidad y redistribución de la demanda. Las conclusiones principales son:

1. La fórmula de scoring multivariable es efectiva como mecanismo redistributivo. La combinación de un 25% de peso en sostenibilidad y un 15% de penalización por congestión —con modificadores multiplicativos en los extremos— produce diferencias de puntuación significativas (hasta 31 puntos en julio) entre destinos saturados y alternativas sostenibles, creando incentivos claros para la redistribución.

2. La explicabilidad es esencial para la adopción. El módulo ExplainabilityEngine garantiza que cada recomendación vaya acompañada de razones comprensibles en lenguaje natural. En sistemas que afectan a decisiones de gasto significativas (un viaje de vacaciones), la transparencia del razonamiento es un factor diferenciador clave frente a las cajas negras.

3. La arquitectura modular facilita la evolución. La separación en submódulos independientes (popularity, sustainability, congestion, confidence, explainability) permite sustituir o mejorar cualquier componente sin afectar al resto del sistema. Esto es especialmente relevante para la incorporación futura de modelos de machine learning más sofisticados.

4. La integración de datos abiertos institucionales aporta legitimidad científica. El uso de datos del INE-EOH y FRONTUR —fuentes estadísticas oficiales del Estado— fundamenta las métricas de congestión en evidencia empírica real, superando la crítica habitual a los sistemas basados exclusivamente en datos propietarios.

5. El sistema RAG abre la puerta a la democratización del acceso. Al permitir que cualquier usuario interactúe con el sistema en lenguaje natural —independientemente de su formación técnica—, el asistente conversacional amplía el alcance del sistema más allá de interfaces tradicionales de formulario.

6. Docker garantiza la reproducibilidad científica. La contenerización completa del stack (backend + frontend) asegura que los resultados presentados sean reproducibles por cualquier evaluador con acceso al repositorio y Docker instalado, alineándose con los principios de ciencia abierta y reproducibilidad.

9.2 Limitaciones

El sistema presenta las siguientes limitaciones que deben ser consideradas al interpretar sus resultados:

Datos sintéticos: Los perfiles de usuarios y el historial de reservas son sintéticos, lo que implica que las puntuaciones de popularidad no reflejan preferencias reales de viajeros. La validación con datos reales de TUI podría modificar significativamente los rankings.

Estática temporal: Los datos de sostenibilidad no se actualizan en tiempo real. La puntuación de sostenibilidad de un destino refleja una instantánea que puede quedar desactualizada ante cambios en la política medioambiental local o en la infraestructura de transporte.

Cobertura geográfica limitada: El catálogo de 20 destinos cubre los principales focos turísticos, pero omite cientos de localidades con potencial turístico no explotado. Una cobertura exhaustiva requeriría un esfuerzo de curación de datos considerablemente mayor.

Fórmula de pesos fija: Los pesos (0.45, 0.25, 0.15, 0.15) son fijos y no se adaptan al perfil de sostenibilidad declarado del usuario. Un usuario con sustainability_preference = High podría beneficiarse de una ponderación dinámica que incremente el peso de sostenibilidad hasta, por ejemplo, 0.35.

RAG sin persistencia: El sistema RAG no mantiene persistencia del historial conversacional entre sesiones. Cada nueva visita del usuario comienza con un contexto vacío.

Ausencia de evaluación A/B: No se ha realizado una evaluación con usuarios reales que compare las decisiones de viaje antes y después de usar Horizon, lo que habría permitido cuantificar el impacto redistributivo real del sistema.

9.3 Líneas futuras

Las líneas de investigación y desarrollo más prometedoras para la evolución del sistema son:

- 1. Ponderación dinámica según perfil de sostenibilidad.** Implementar una variación de la fórmula que ajuste los pesos en función del nivel de `sustainability_preference` declarado por el usuario, creando una experiencia más personalizada.
- 2. Integración de datos climáticos (AEMET).** La API de AEMET (Agencia Estatal de Meteorología) proporciona predicciones meteorológicas que podrían enriquecer las recomendaciones con información sobre condiciones climáticas esperadas en el mes de viaje seleccionado. La clave `AEMET_API_KEY` ya está preparada en la arquitectura Docker.
- 3. Motor de filtrado colaborativo híbrido.** Complementar el filtrado por contenido actual con un componente de filtrado colaborativo basado en las reservas históricas, utilizando técnicas de factorización matricial (ALS, SVD) o redes neuronales (NCF) para capturar preferencias implícitas.
- 4. Actualización dinámica de datos.** Desarrollar pipelines de ingesta periódica que actualicen los datos de ocupación hotelera desde la EOH del INE y los flujos internacionales desde FRONTUR, manteniendo las métricas de congestión sincronizadas con la realidad estadística.
- 5. Evaluación con usuarios reales (A/B testing).** Diseñar un experimento controlado con viajeros de TUI en el que un grupo use Horizon y otro el sistema de recomendación estándar, midiendo el porcentaje de reservas realizadas en destinos menos congestionados como métrica de impacto redistributivo.
- 6. Expansión a Portugal y Marruecos.** TUI Care Foundation opera en toda la región EMEA. La arquitectura modular de Horizon facilitaría la extensión del catálogo a destinos portugueses y marroquíes, donde la problemática de concentración turística presenta dinámicas similares (Algarve, Marrakech).
- 7. Certificación de carbono por destino.** Integrar datos de emisiones de CO₂ por modo de transporte y tipo de alojamiento para calcular una huella de carbono estimada por viaje, añadiendo una dimensión de transparencia ambiental cuantificada.

10. Bibliografía

- Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
- Gretzel, U., Fesenmaier, D. R., & O’Leary, J. T. (2006). The transformation of consumer behaviour. In D. Buhalis & C. Costa (Eds.), *Tourism Business Frontiers* (pp. 9–18). Elsevier.
- Instituto Nacional de Estadística (INE). (2024). *Encuesta de Ocupación Hotelera (EOH): Metodología y resultados*. https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736177015
- Instituto Nacional de Estadística (INE) — FRONTUR. (2024). *Movimientos Turísticos en Fronteras: Cifras 2024*. https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736176996
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender Systems Handbook* (pp. 73–105). Springer. https://doi.org/10.1007/978-0-387-85820-3_3

Organización Mundial del Turismo (OMT). (2023). *Recommendations on Sustainable Tourism Indicators*. UNWTO Publications. <https://www.unwto.org/sustainable-development>

Secretaría de Estado de Turismo. (2024). *Plan de Sostenibilidad Turística en Destinos 2021–2025: Informe de seguimiento*. Ministerio de Industria y Turismo de España.

TUI Care Foundation. (2024). *Future Shapers Spain: Reto de turismo sostenible*. Documento interno del programa.

Unión Europea. (2016). *Reglamento General de Protección de Datos (RGPD)* — Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo. Diario Oficial de la Unión Europea.

11. Anexos

Anexo A: Tabla de endpoints de la API REST

Método	Endpoint	Descripción	Request Body	Response
GET	/	Root endpoint	—	{"application": str, "status": str}
GET	/health	Health check	—	{"status": "ok"}
POST	/recommendations	Genera recomendaciones	RecommendationRequest	{"recommendations": List[Recommendation]}
GET	/users	Lista todos los user_id	—	{"user_ids": List[str]}
GET	/users/{user_id}	Perfil de un usuario	—	Campos del perfil de usuario
POST	/chat	Consulta conversacional RAG	{"message": str, ...}	{"response": str}

Modelo RecommendationRequest (Pydantic):

```
class RecommendationRequest(BaseModel):
    user_id: str          # Ej: "U001"
    month: int            # 1-12
    top_n: int = 5        # Número de resultados (default: 5)
```

Modelo de recomendación individual en la response:

```
{
    "destination_id": "D019",
    "destination_name": "Picos de Europa",
    "final_score": 88.86,
    "preference_score": 100.0,
    "popularity_score": 52.10,
    "sustainability_score": 72.10,
```

```
"congestion_score": 8.0,  
"confidence_score": 81.68,  
"explanations": [  
    "Strong match for your Nature travel preferences.",  
    "Lower expected congestion than comparable destinations."  
],  
"recommendation_rank": 1  
}
```

Códigos de estado HTTP:

Código	Situación
200 OK	Petición procesada correctamente
404 Not Found	Usuario no encontrado (GET /users/{user_id})
422 Unprocessable Entity	Request body inválido (validación Pydantic)
500 Internal Server Error	Error inesperado del motor

Anexo B: Esquemas de los CSV

B.1 destinations.csv — Esquema completo

Campo	Tipo Python	Ejemplo
destination_id	str	“D001”
destination_name	str	“Mallorca”
region	str	“Balearic Islands”
destination_type	str	“Beach”
beach_score	int (1–10)	10
culture_score	int (1–10)	7
nature_score	int (1–10)	8
nightlife_score	int (1–10)	8
family_friendly_score	int (1–10)	9
avg_price_per_day	float	145.0

B.2 users.csv — Esquema completo

Campo	Tipo Python	Ejemplo
user_id	str	“U001”
country	str	“Germany”
age_group	str	“18-24”
budget_level	str	“Low”
travel_style	str	“Nature”
sustainability_preference	str	“Medium”

B.3 bookings_history.csv — Esquema completo

Campo	Tipo Python	Ejemplo
booking_id	str	“B0001”
user_id	str	“U082”

Campo	Tipo Python	Ejemplo
destination_id	str	“D001”
booking_date	str (date)	“5/21/2025”
travel_month	int (1–12)	4
stay_days	int	6
total_price	float	819.42
user_rating	int (1–5)	4

B.4 sustainability_scores.csv — Esquema completo

Campo	Tipo Python	Ejemplo
destination_id	str	“D001”
carbon_score	float	52.0
local_business_score	float	60.0
public_transport_score	float	45.0
sustainability_score	float	52.7

B.5 congestion_scores.csv — Esquema completo

Campo	Tipo Python	Ejemplo
destination_id	str	“D001”
month	int (1–12)	7
congestion_score	float (0–100)	100.0
congestion_level	str	“Very High”

B.6 Clasificaciones del sistema

Sostenibilidad: | Umbral | Clasificación | Modificador | |—|—|—| | ≥ 85 | Excellent | $\times 1.05$ (+5%) | | ≥ 70 | Good | $\times 1.00$ | | ≥ 50 | Moderate | $\times 1.00$ | | < 50 | Poor | $\times 0.90$ (–10%) |

Congestión: | Umbral | Clasificación | Modificador | |—|—|—| | < 40 | Low | $\times 1.05$ (+5%) | | 40–70 | Moderate | $\times 1.00$ | | 70–80 | High | $\times 1.00$ | | > 80 | Very High | $\times 0.90$ (–10%) |

Confianza: | Rango | Interpretación | |—|—| | 80–100 | Alta confianza — destino muy alineado con el perfil | | 60–79 | Confianza media — buen ajuste general | | 40–59 | Confianza moderada — ajuste parcial | | < 40 | Confianza baja — exploración fuera del perfil habitual |